M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_14

14. Extension Development and Deployment

Matt Frisbie[1]

(1) California, CA, USA

The process of developing, testing, and publishing a browser extension is significantly different from developing a traditional website. The steps involved more closely resemble that of a mobile app than a web technology. Gaining a better understanding of how to effectively develop, publish, and distribute your extension is critical to mastery of the subject.

> **Note** This chapter will be Google Chrome-centric, but all major browsers (with the exception of Safari) offer nearly identical facilities when developing locally and when publishing to a marketplace.

Local Development

You'll spend quite a lot of time developing your extension locally, so it's worth getting familiar with how all the parts fit together. In this section, we'll explore how a locally loaded extension works and all the different ways you can look under the hood.

> **Tip** For developers new to extension development, the *Browser Extension Crash Course* chapter demonstrates the basic process of loading a local extension into your browser for development purposes.

Inspecting Your Extension

Begin by installing this dummy extension:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "options_ui": {
    "open_in_tab": true,
    "page": "options.html"
  },
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "css": [],
      "js": ["content-script.js"]
    }
  ],
  "permissions": [
    "scripting",
    "declarativeNetRequest",
    "tabs"
   ],
  "host_permissions": ["<all_urls>"]
}
```

Example 14-1a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Options</h1>
    <script src="options.js"></script>
  </body>
</html>
```

Example 14-1b options.html

```
console.log("Initialized options!");
```

Example 14-1c options.js

```
console.log("Initialized background!");
```

Example 14-1d background.js

```
console.log("Content script initialized!");
```

Example 14-1e content-script.js

Once this extension is loaded locally, your browser will add a card inside the browser extension management view. In Google Chrome, the URL for this page is `chrome://extensions`. An example card is shown in Figure 14-1.

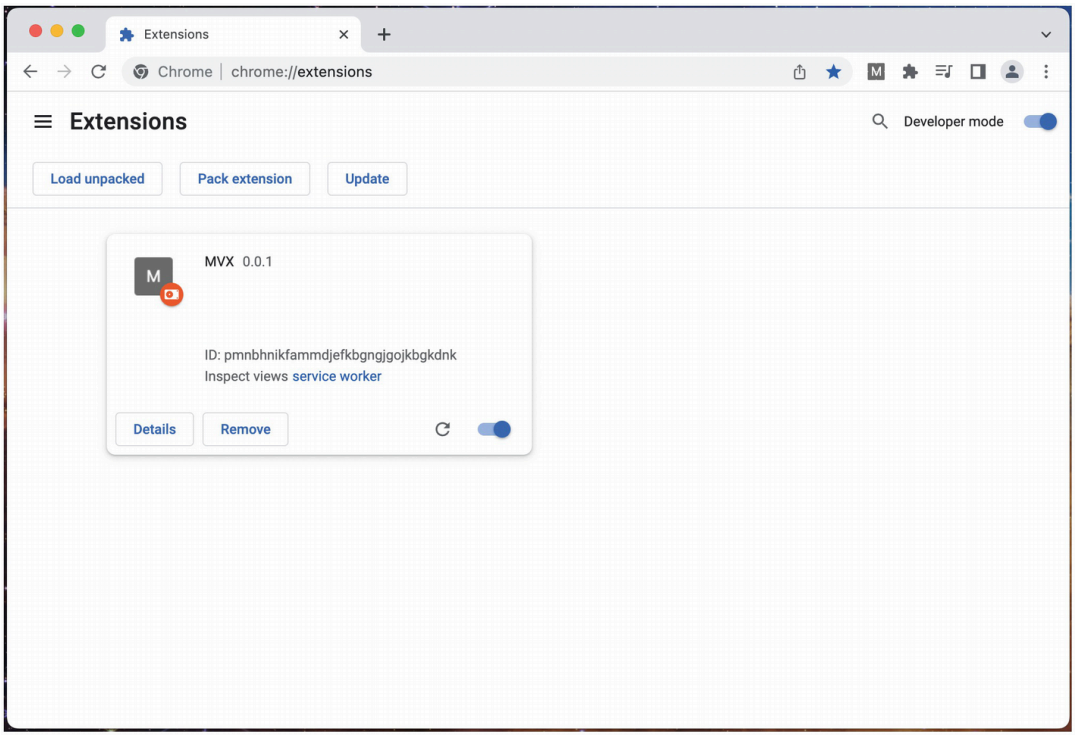

Figure 14-1 An example extension card in the extension management page view

From this card, you're able to view the details page, uninstall or disable the extension, view the extension's errors page, and reload the extension.

Note Details about reloading an extension are covered later in this chapter.

Clicking to open the details page will reveal something like Figure 14-2.

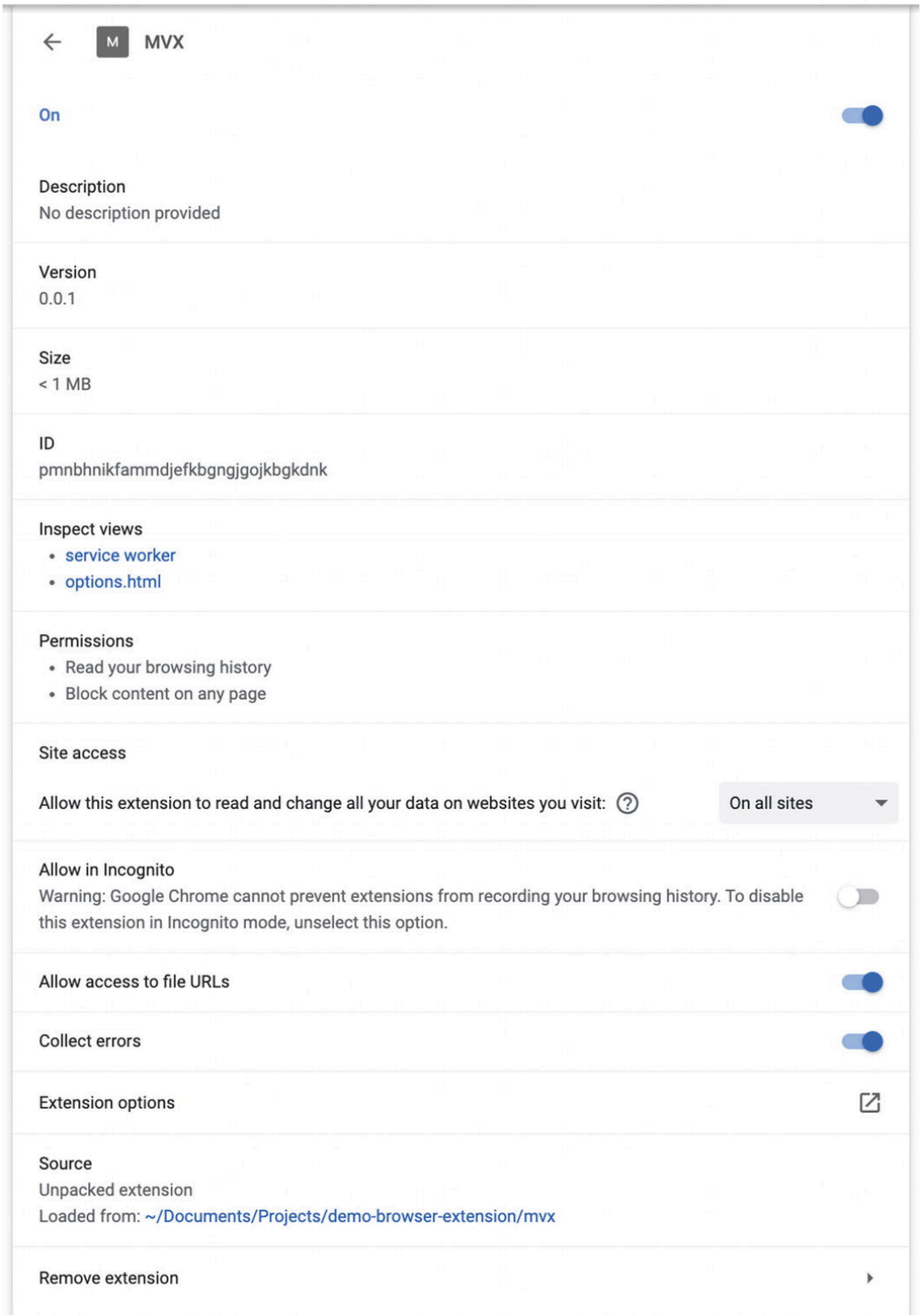

Figure 14-2 An example extension detail view

There are a few things to note in this view:

- **This view is a snapshot.** Values listed inside are not real-time and will not update unless the page reloads.
- **Inspect views** contains a list of links paired to each extension view that is currently active. This list is analogous to the values returned from `chrome.extension.getViews()`. Each link will open a developer tools window targeted at that particular extension view. (The "views" name can be confusing, as this tool also allows you to inspect the background service worker, which has no user interface.)
- **Permissions** displays the list of permissions prompts that the extension would have to explicitly request if it were loaded in production. Because this extension is loaded in development mode, the permissions dialog is silenced.
- **Source** indicates where in the current OS's filesystem it is loading the extension files from.

Inspecting an Extension View

Clicking the "options" link under *Inspect Views* will open the corresponding developer tools interfaces (Figure 14-3).

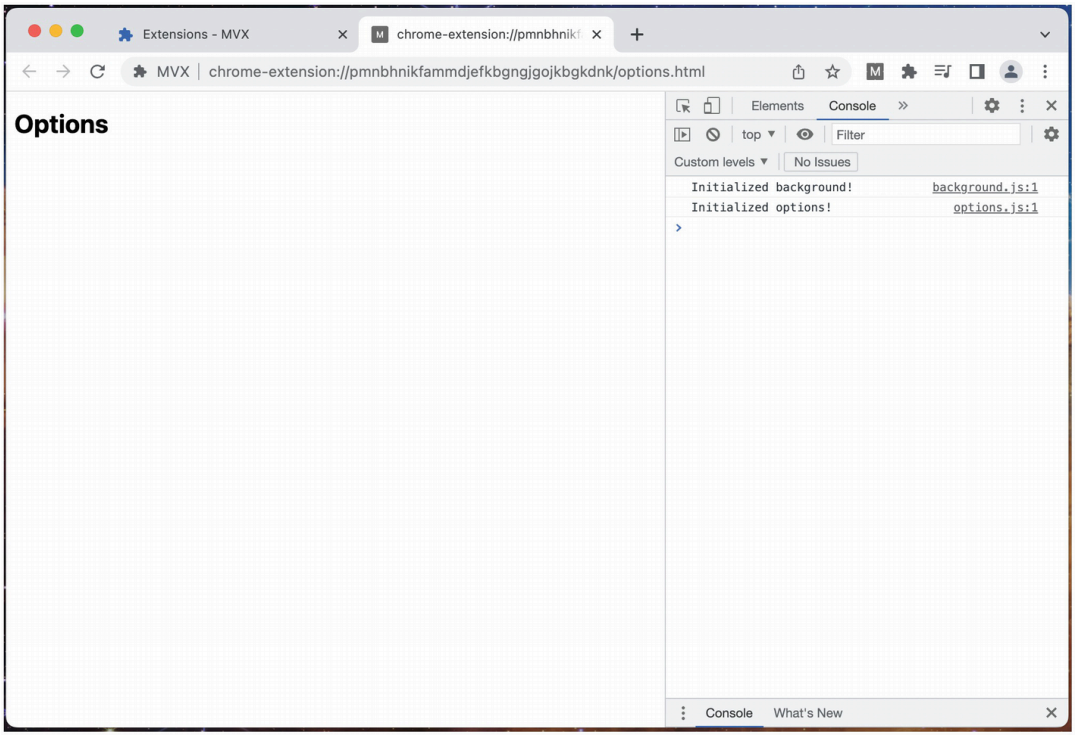

Figure 14-3 The options page inspect view

Note here that the console is showing log output for both the current tab console output and the background service worker. If you wish to split out console outputs, the developer console allows you to select specific sources (Figure 14-4).

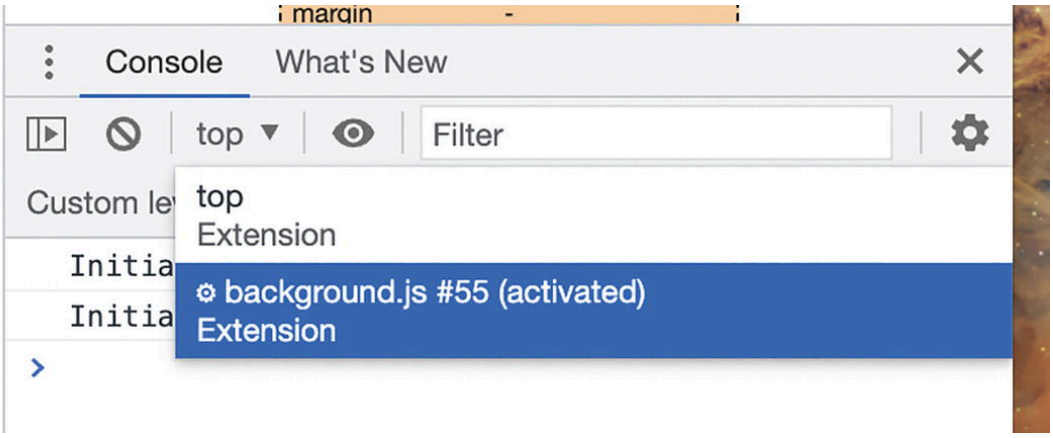

Figure 14-4 Selecting a logging source in developer tools

Inspecting a Background Service Worker

It's also possible to directly inspect a background service worker. Clicking the *Inspect view* link will reveal something like the following (Figure 14-5).

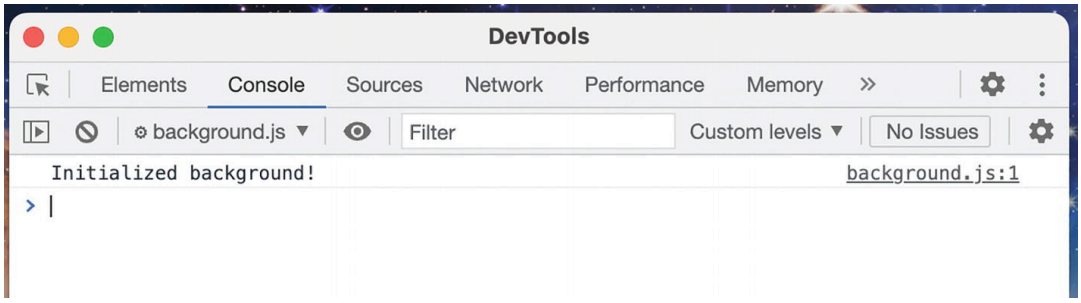

Figure 14-5 Inspecting the background service worker

WarningOne *extremely* important thing to keep in mind is that this background service worker will interfere with the lifecycle of the service worker. It will prevent the service worker from going idle. It will also prolong the lifespan of a service worker past reloads, causing bizarre behavior when debugging an extension. **When you are done with a service worker developer tools window, close it immediately**. Leaving it open in the background will change the behavior of

your local extension in unpredictable ways and make it much harder to debug.

Chrome offers a real-time view of your service worker status and log output at `chrome://serviceworker-internals/` (Figure 14-6). This page will not interfere with how your service workers behave, allowing them to go idle.

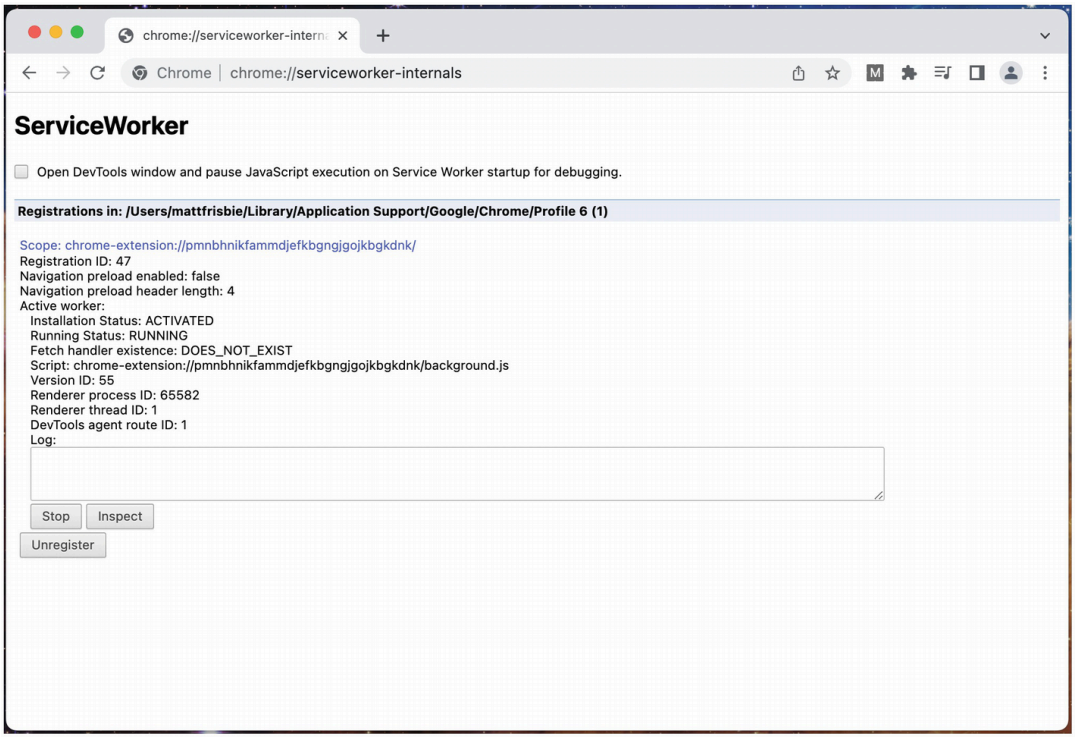

Figure 14-6 Google Chrome's service worker internals page

Inspecting a Content Script

Inspecting the console for web pages with content scripts will mix the console output for the web page and the content script together (Figure 14-7).

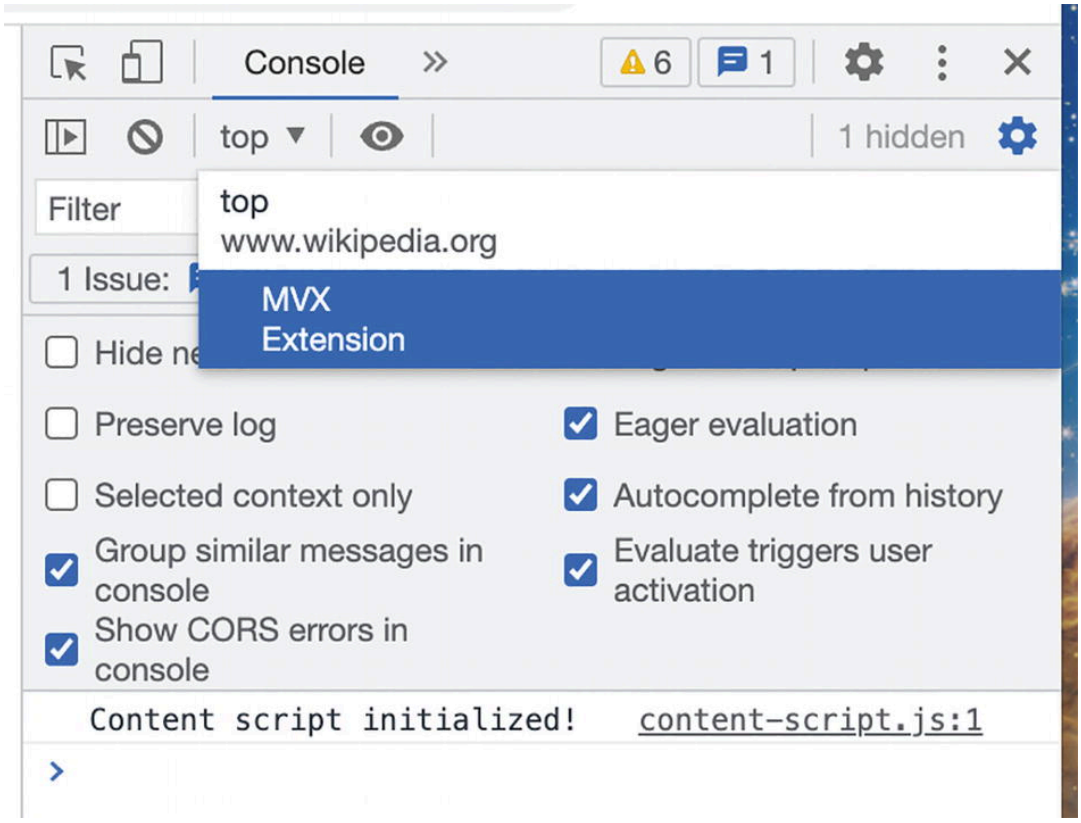

Figure 14-7 Example console output for Wikipedia.org

As with background service workers and options pages, it is possible to filter for only the content script console output.

File Changes

In the extension detail view shown above, it indicates where in the filesystem the extension was loaded from. This is because the extension is serving files directly from that directory! Any changes you make to the source files will be immediately reflected the next time the extension loads them via network request. For example, a modification to `popup.html` will show the very next time the popup is opened. This does *not* apply to files that the extension uses only on install such as `manifest.json`. Updates to these files will only be reflected upon a formal extension reload.

Error Monitoring

Monitoring errors during development can be tricky. For views that have HTML interfaces, errors thrown in the page can be monitored via the normal

developer tools console. All errors thrown in an extension, no matter where they are thrown, will appear in the extension error view (Figures 14-8, 14-9 and 14-10).

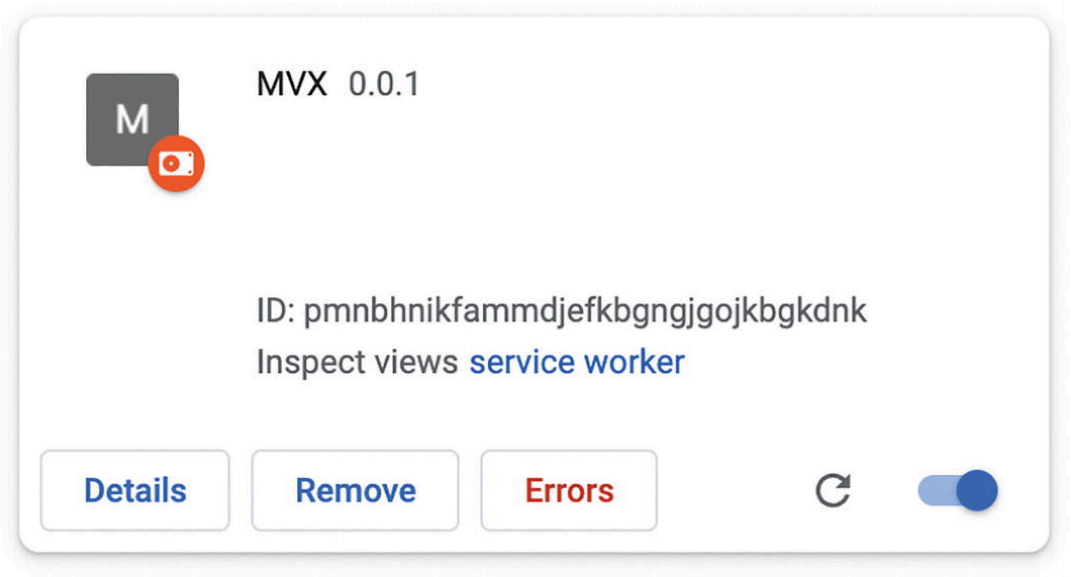

Figure 14-8 The Errors button will only appear when the extension throws its first error. This button will open the extensions error view

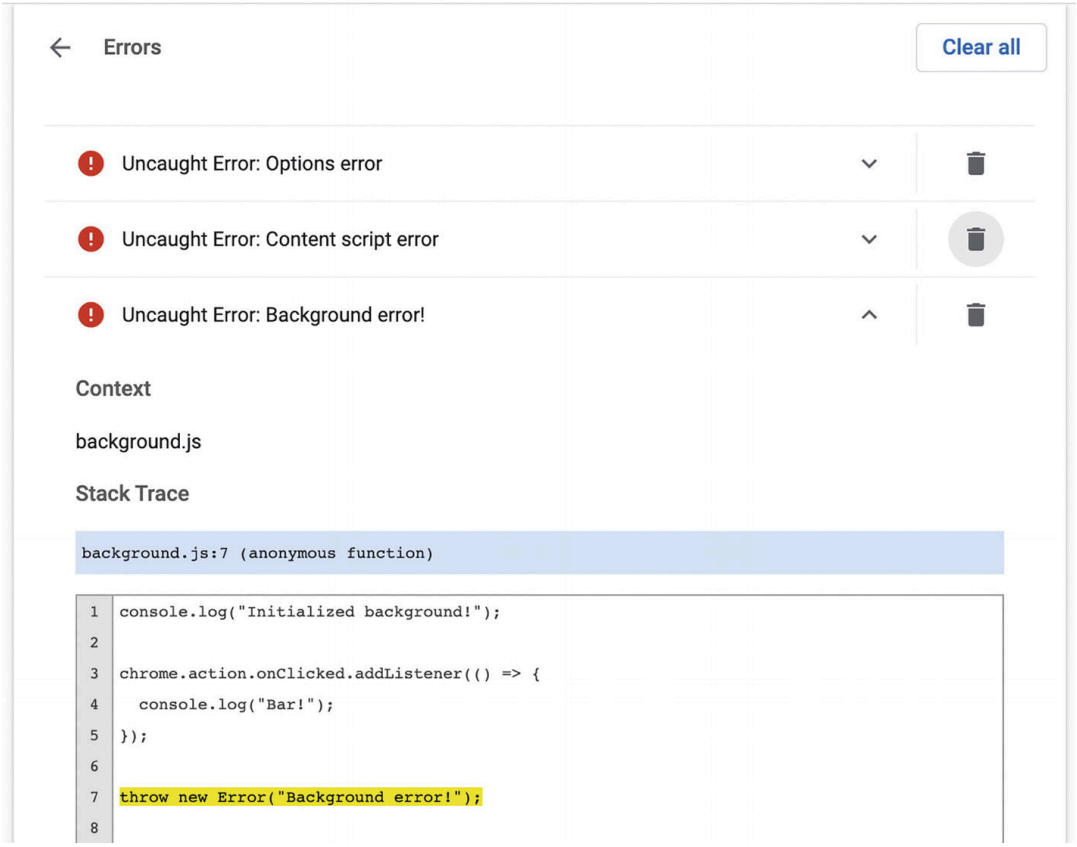

Figure 14-9 The extension error view showing errors from multiple sources

Warning If a service worker throws an error in the first turn of the event loop, it will fail to register. This is important, as any event handlers that it sets will never execute.

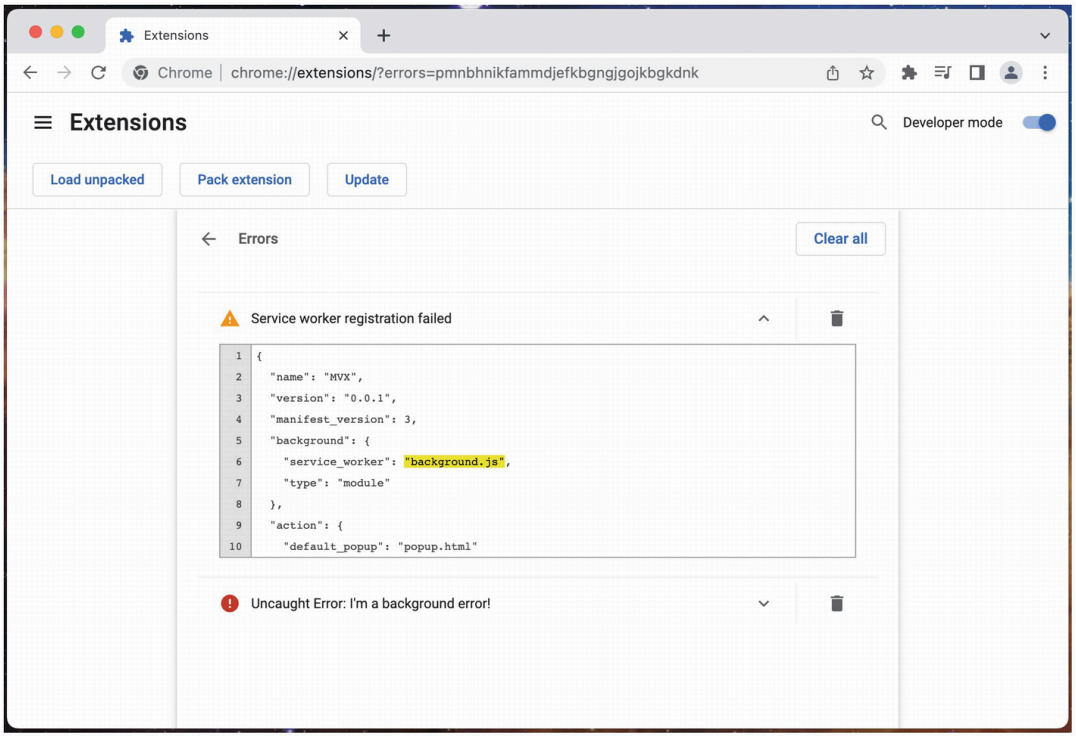

Figure 14-10 A service worker fails to register after an error thrown during initialization

Extension Reloads

There are several different points at which portions of the extension are reloaded:

- An **extension reload** obtains a new copy of `manifest.json`, updates the background service worker, and closes any stale popups and options pages that are open. This is required when the `manifest.json` changes.
- An **extension page reload** is a page refresh of any page that uses the extension protocol `chrome-extension://`. This is required to reflect changes in HTML, JS, CSS, and images in popup and options pages.
- A **web page reload** is a page refresh of any web page that has content scripts injected. This is required to reflect changes in the in-

jected content scripts.

- A **devtools reload** is closing and opening the browser's developer tools interface. This is required to reflect changes in devtools pages. There are three ways to force an extension reload:

- Uninstall and reinstall the extension
- Click the reload icon ↺ in the corresponding card on the Chrome Extensions page
- Programmatically reload the extension using `chrome.runtime.reload()` or `chrome.management.setEnabled()`

Automated Extension Tests

Because the components of browser extensions are built from web technologies, testing them is not too different from testing web pages. Popular tools such as Jest and Puppeteer can be configured to power automated tests for your extension. Some idiomatic aspects of browser extensions such as popups and content scripts can't be directly tested, but with some clever workarounds you will still be able to compose a robust and effective test suite.

> **Note** This section assumes you are familiar with JavaScript automated testing. If not, the Jest documentation has a good tutorial geared for beginners: **https://jestjs.io/docs/tutorial-react**

Unit Tests

Because unit tests don't rely on extension components rendering inside their native browser containers, these tests can be written in a nearly identical fashion to regular web page unit tests. To stub out the chrome extension APIs, the `sinon-chrome` package is excellent: **https://github.com/acvetkov/sinon-chrome**. This package allows you to stub out the callback values for extension APIs in a very clean manner.

A popular unit test setup for extensions is to use Parcel as the main build tool, React as the framework, and Jest as the test runner. The following example sets up a very simple extension with a popup that reads from the chrome storage API. The example includes some unit tests that make sure the popup renders as expected:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "action": {
    "default_popup": "index.html"
  },
  "permissions": ["storage"]
}
```

Example 14-2a manifest.json

```
{
  "scripts": {
    "start": "parcel watch manifest.json –host localhost",
    "build": "parcel build manifest.json",
    "test": "jest –watch"
  },
  "dependencies": {
    "@types/chrome": "^0.0.196",
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  "devDependencies": {
    "@babel/preset-env": "^7.18.10",
    "@babel/preset-react": "^7.18.6",
    "@parcel/config-webextension": "^2.7.0",
    "@testing-library/jest-dom": "^5.16.5",
    "@testing-library/react": "^13.4.0",
    "jest": "^29.0.2",
    "jest-environment-jsdom": "^29.0.2",
    "parcel": "^2.7.0",
    "process": "^0.11.10",
    "sinon-chrome": "^3.0.1"
```

```
  },
  "jest": {
    "testEnvironment": "jsdom"
  }
}
```

Example 14-2b package.json

```
<!DOCTYPE html>
<html>
  <body>
    <div id="app"></div>
    <script type="module" src="index.tsx"></script>
  </body>
</html>
```

Example 14-2c index.html

```
import React from "react";
import { createRoot } from "react-dom/client";
import { Popup } from "./Popup";
const rootElement = document.getElementById("app");
const root = createRoot(rootElement);
root.render(<Popup />);
```

Example 14-2d index.tsx

```
{
  "extends": "@parcel/config-webextension",
  "transformers": {
    "*.{js,mjs,jsx,cjs,ts,tsx}": [
      "@parcel/transformer-js",
      "@parcel/transformer-react-refresh-wrap"
    ]
  }
}
```

Example 14-2e .parcelrc

```
{
```

```
  "presets": [
    ["@babel/preset-env", { "targets": { "node": "current" } }],
    "@babel/preset-react"
  ]
}
```

Example 14-2f .babelrc

```
import React, { useEffect, useState } from "react";
export function Popup() {
  const [count, setCount] = useState(null);
  useEffect(() => {
    chrome.storage.sync.get(["count"], (result) => {
      if (typeof result.count !== "number") {
        result.count = 0;
      }
      setCount(result.count);
    });
  }, []);
  if (count != null) {
    return <h1>Popup count: {count}</h1>;
  }
}
```

Example 14-2g Popup.tsx

```
import { render, screen } from "@testing-library/react";
import React from "react";
import { Popup } from "./Popup";
import chrome from "sinon-chrome";
describe("Popup", () => {
  beforeAll(() => {
    global.chrome = chrome;
  });
  test("Reads from the storage api", () => {
    render(<Popup />);
    expect(chrome.storage.sync.get.withArgs(["count"]).calledOnce).toBe(true);
  });
  test("Renders a default value", () => {
    chrome.storage.sync.get.withArgs(["count"]).yields({ count: undefined });
    render(<Popup />);
```

```
      expect(screen.queryByText("Popup count: 0")).not.toBeNull();
    });
    test("Renders the storage API value", () => {
      chrome.storage.sync.get.withArgs(["count"]).yields({ count: 3 });
      render(<Popup />);
      expect(screen.queryByText("Popup count: 3")).not.toBeNull();
    });
    afterAll(() => {
      chrome.flush();
    });
  });
```

Example 14-2h Popup.test.js

> **Tip** If you are setting this example up from scratch, running `yarn install` should get this setup working.

For developers already familiar with how typical React unit testing looks, most of this should not surprise you. Some things to note about this implementation

- Most of this example is typical unit test boilerplate. The parts that are unique to browser extension unit testing are shown in bold.
- Parcel does not need Babel to compile, but Jest does. The extra values in `.parcelrc` are to conditionally use Babel as a compiler only for unit tests.
- The `sinon-chrome` package is being used to detect calls to the storage API, but also is able to stub out the values that are returned in the API's callback.

With all the packages installed, you should find that `npm run start` will start up a Parcel development build, and `npm test` will run your unit test suite (Figure **14-11**).

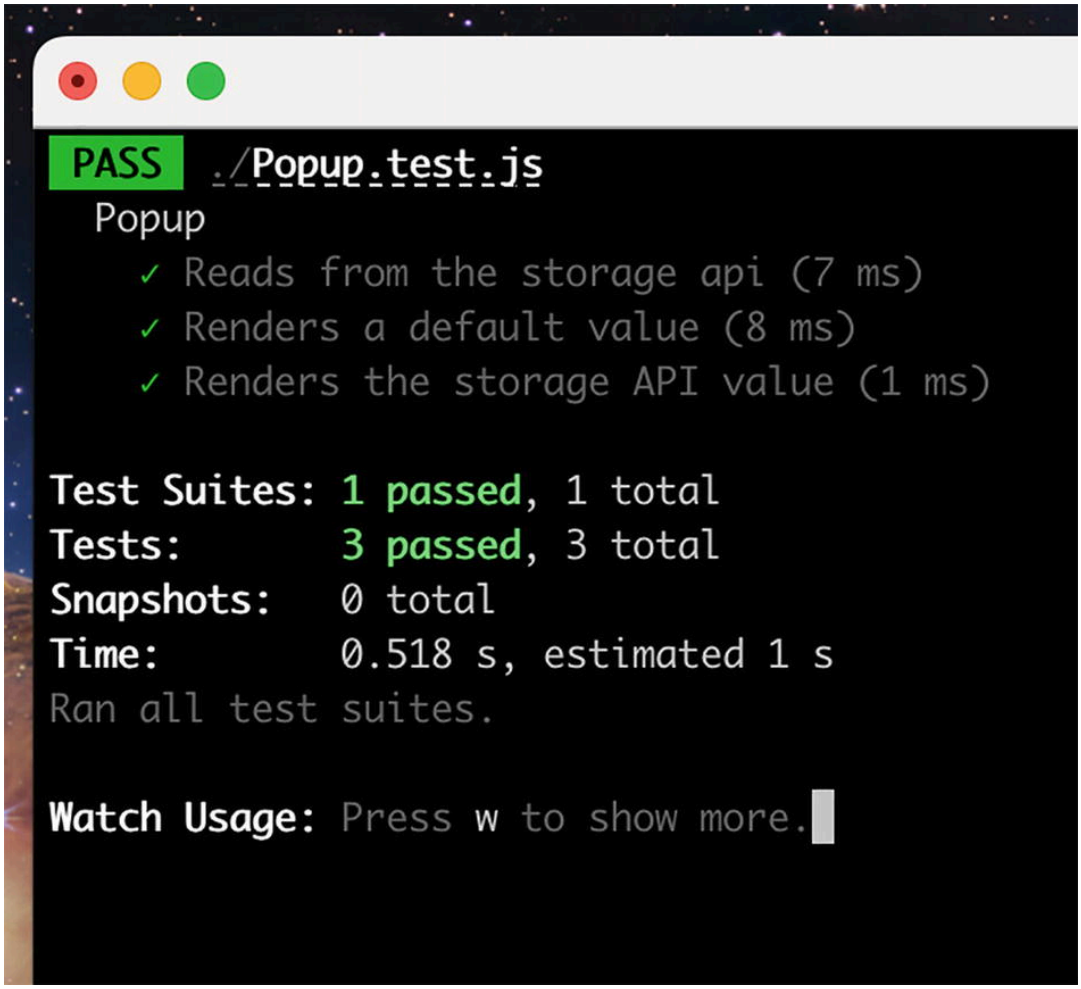

Figure 14-11 A passing unit test run

Integration Tests

Integration testing browser extension is made possible via Puppeteer (**https://pptr.dev/**). It allows you to programmatically control a Chromium browser, including navigating to URLs and dispatching user events. It cannot perfectly replicate all the different ways that a browser extension will render its views, but it is by far the best tool available to power an integration test suite.

Puppeteer can be configured to load an extension via the `--load-extension` flag. This flag should be pointed to a local directory on your machine that contains your extension code. You will also need to disable headless mode, as extension cannot be loaded in a headless browser:

```
const path = `path/to/your/extension`;
const browser = await puppeteer.launch({
```

```
    // Disable headless mode
    headless: false,
    args: [
      `--disable-extensions-except=${path}`,
      `--load-extension=${path}`
    ]
  });
```

This will boot up a Chromium browser, but you'll need to direct the browser to a URL for your extension. Of course, you will need to know ahead of time what the extension's ID is. A good strategy for this is to control the extension ID via the manifest's `key` field:

```
const page = await browser.newPage();
await page.goto(
  'chrome-extension://<extensionID>/popup.html');
// Now you can run test code
```

Because all extension views, including popup pages, can be loaded via direct extension URL, all your extension pages can be tested in this way. Puppeteer can be used to run any Chromium browser, so an integration test suite can target browsers like Google Chrome, Microsoft Edge, or Opera.

> **Note** Setting up a full working integration test with puppeteer is outside the scope of this book. Some full examples are listed in the *Additional Reading* section below.

Additional Reading

Setting up automated testing is tricky, and frankly, annoying. The following are some links to blog posts and documentation about various extension testing setups:

- https://www.streaver.com/blog/posts/testing-web-extensions
- https://github.com/clarkbw/jest-webextension-mock

- https://www.npmjs.com/package/jest-chrome
- https://medium.com/information-and-technology/unit-testing-browser-extensions-bdd4e60a4f3d
- https://medium.com/information-and-technology/integration-testing-browser-extensions-with-jest-676b4e9940ca
- https://gokatz.me/blog/automate-chrome-extension-testing/
- https://tweak-extension.com/blog/complete-guide-test-chrome-extension-puppeteer/
- https://pptr.dev/#working-with-chrome-extensions

Publishing Extensions

To publish an extension to the Chrome Web Store, you'll first need to pay the one-time $5 fee for a developer account. Once this account is set up, you're ready to submit. Extensions are always uploaded as a zip file of your extension's files and assets.

Store Listing

To publish an extension, you'll need to specify how it should appear to users in the Chrome Web Store. Some items are automatically extracted from the manifest:

- Extension title
- Extension summary
- Icon

Everything else, you'll need to supply yourself:

- Description
- Extension Category
- Assets (including videos and screenshots)
- URLs (main website, support URL, privacy policy URL)

> **Note** These fields can be changed later, but each change will make the entire extension undergo a slow manual review – even if the code was not changed.

Privacy Practices

In the Chrome Web Store, you will need to provide a high-level description of what this extension is doing, as well as a justification for each permission category that is considered sensitive (identity, scripting, etc.). You will also need to indicate if and how you are tracking your users, and in what way.

Review Process

Your extension will undergo manual review the first time it is submitted to ensure it meets Chrome Web Store standards. Once it's approved, it will be available for anyone to install!

Updating Extensions

Once your extension is published, the process of updating it is more or less identical to the initial publishing process. You'll upload a zip file with an updated version number in the manifest. Any additional permissions will require additional justification.

Update Considerations

Once you have an existing userbase and want to push out an update, there are some things you should keep in mind.

Update Delays

Updates are not rolled out immediately. Once you submit an update, it must be reviewed and approved before a user's browser will be able to install it. The user's browser will periodically send out update checks

every few hours and download the update if there is one available. However, the browser will lazily apply that update if the extension is active, so there is sometimes a considerable delay between when an update is published and when it is installed. This delay can be between a few minutes and upwards of a week.

Auto-Disabling

Suppose you have a collection of users with your extension installed. If a new version is published that requires a new permission that needs explicit user approval, your extension will be disabled until the user grants that permission. The browser will not pop up a dialog when the extension is disabled, only a small icon will appear. This means that you may suffer unintentional user attrition when adding addition permissions. Using optional permissions can solve this problem.

> **Note** Details of this are covered in detail in the *Permissions* chapter.

Cancelling Updates

When this book was written, it was unfortunately still not possible to cancel an update once it is submitted for review. Be very careful when submitting for review, as a mistake may mean you must wait days before a fix can be submitted.

Automated Update Publishing

Instead of manually uploading zip files to a web page, it is possible to automatically submit your update to the Chrome Web Store via a REST API. To do this, you will first need to acquire credentials to authenticate your API requests. Details of this process can be found here:

https://developer.chrome.com/docs/webstore/using_webstore_api/

Once you have the required credentials, you can manage a significant portion of your extension via API, documented here:

https://developer.chrome.com/docs/webstore/api_index/

Interacting with the API via command line is clunky. Instead, use an NPM package to push your updates:

https://github.com/simov/chrome-webstore

> **Tip** The Plasmo platform has a terrific automatic submission tool called Browser Platform Publisher. Refer to the *Tooling and Frameworks* chapter for details.

Tracking User Activity

Once you have people installing and using your extension, you will certainly want to know how many of them there are and what they are doing.

Dashboard Metrics

The Chrome Developer Dashboard shows you several important metrics about your extension, including:

- Users/day
- Impressions/day
- Installs/day
- Uninstalls/day

> **Note** The number of weekly users you see in the Chrome Web Store is the amount of users whose Chrome browser has checked for an update of your app within the last week. It is not the number of people who have installed your item.

Analytics Libraries

Tracking user activity in a browser extension is mostly the same as tracking web page activity. There are a few wrinkles to consider:

- **Events in a content script should be tracked in a background service worker.** Outgoing network requests to send analytics data are subject to adblockers and cross-origin restrictions.
- **Events in a background script will not have a page URL.**
- **You must preload your analytics library scripts in manifest v3.** Many analytics libraries such as Google Analytics give the option to dynamically load an analytics script – this is no longer allowed.

> **Tip** Adblockers will be unable to block requests sent from extension pages or background scripts. Therefore, unlike web pages (where a large fraction of analytics traffic gets blocked), you can expect to have near-perfect analytics fidelity with browser extensions.

Setting Up Google Analytics

Many extension developers wish to use Google Analytics (GA) for their browser extension. GA4 makes this a bit trickier, but with some configuration, this is certainly possible.

Set Up ga.js

For a manifest v3 extension, you will need to download the `ga.js` script (**`https://www.googletagmanager.com/gtag/js?id=GA_TRACKING_ID`**) and save it as a local script inside your extension.

Google Analytics checks the URL protocol of the active page before deciding to send analytics pings. If it isn't `http` or `https`, it will decline to send. When using the GA3 script, it was possible to override this behavior via `checkProtocolTask`:

```
ga('set', 'checkProtocolTask', null);
```

However, it appears that in GA4, there is currently no ability to override this behavior. If you wish to use GA4 with an extension, you will need to disable this protocol check. Find the following line in your `ga.js` and remove it or comment it out:

```
"http:" != c && "https:" != c && (N(29), a.abort());
```

After doing this, you should be able to set up Google Analytics as follows:

```
const script = document.createElement("script");
script.async = true;
script.src = "/ga.js";
document.body.appendChild(script);
window.dataLayer = window.dataLayer || [];
function gtag() {
  dataLayer.push(arguments);
}
const GA_ID = "GA_TRACKING_ID";
gtag("js", new Date());
// Disable automatic pageview reporting
gtag("config", GA_ID, {
  send_page_view: false,
});
// Manually send the pageview event
gtag("event", "page_view", {
  page_path: window.location.path,
});
```

This is a bit of a hack, but at the time this book was published it is the only known way of getting the GA4 script to work nicely with browser extensions.

Install and Uninstall Events

You are able to perform special tasks when the user installs and uninstalls an extension. For example, the background script can perform a task only when the user newly installs an extension:

background.js

```
chrome.runtime.onInstalled.addListener((details) => {
    if (details.reason === chrome.runtime.OnInstalledReason.INSTALL) {
      // Anything in here will only execute on first install
      openWelcomePage();
    }
});
```

You won't be able to execute code on an uninstall, but you are able to direct the user to a URL of your choosing with `chrome.runtime.setUninstallURL`:

```
chrome.runtime.setUninstallURL("https://foobar.com/survey");
```

There are no restrictions on this URL, I recommend using it to collect analytics data, give a survey, display contact information for troubleshooting, or offer some helpful tips that might get them to reinstall.

Summary

In this chapter, we discussed a number of topics involved with building and publishing an extension. We discussed a range of tactics that can be used to better understanding what is going on inside your local extension. Next, we went through the different testing formats that work well for browser extensions. We went through all the important parts involved with publishing and updating extensions in the Chrome Web Store. Finally, we discussed how analytics tools can be effectively folded into your browser extension.

In the next chapter, we will cover all the details involved with building a browser extension that targets more than one browser.